CSC-302: Programming in Python

Unit-I: *Lambda Functions*



Lambda functions

- **Lambda function** or ***Anonymous functions*** are single line function definitions
- Lambda functions are **usually only called once** even though they **can be called multiple times by assigning a label**
- Syntax

```
label = lambda param(s): exp
```

- A lambda can only contain a single expression (no statements or multi-line bodies)



Usage examples

```
square = lambda x: x * x

print(square(2))    # 4
```

```
area_rect = lambda l, b: 2 * (l + b)

print(area_rect(2.0, 4.0))  # 12.0
```

```
members = ['john lennon', 'paul mccartney', 'ringo starr', 'george harrison']
members.sort(key=lambda name: name.split()[-1])
print(members)    # ['george harrison', 'john lennon', 'paul mccartney',
'ringo starr']
```



Lambda functions as return value

- **A function can build and return a lambda, which makes it a *function factory***: it produces customized functions on demand
- **The returned lambda “remembers” values from the enclosing function’s scope**, even after function factory has finished running
- This remembered environment is called a ***closure***
- **Each call creates a separate closure**



Usage example

```
def raise_by(n):
    return lambda m: m**n

raise_by_two = raise_by(2)
raise_by_five = raise_by(5)

print(raise_by_two(3))  # 9
print(raise_by_five(4)) # 32
```

```
def price_calc(start,hourly_rate):
    return lambda hours: start + hourly_rate * hours

walkin_cost = price_calc(10,30)
premium_cost = price_calc(1,25)

print(walkin_cost(2))  # 70
print(premium_cost(2))  # 51
```



Calling Lambda functions immediately

- We can follow the syntax below to immediately invoke the lambda function as soon as it is declared

  `(lambda param(s): exp)(param(s))`

- For example

```
print(f'circle of radius 3.8 has area {(lambda pi, r: pi*r**2)(3.14, 3.8)}')
```



Opening a file using `with`

- The advantage of using **`with`** clause is that **any file that is opened using this clause is closed automatically, once the control comes outside the `with` clause**
- Hence **if the user forgets to close the file explicitly or if an exception occurs, the file is closed automatically**
- Syntax

```
with open (file_name, access_mode) as file_object:
```



Usage example

```
with open('lyrics.txt', 'r') as f:
    content = f.read()
    print(content)
```

